INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 1

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica treballarem amb el programa Ripes, un programa que treballa amb llenguatge assemblador, que és posteriorment traduït a màquina. En aquest programa, podrem veure el comportament de les adreces de memòria i els seus valors emmagatzemats. Dins d'aquest programa, veiem que Ripes ofereix dos tipus de processadors: el RSCP i el R5SP. Al llarg d'aquesta pràctica, s'anirà treballant amb les diferències que presenta un i l'altre.

Per altra banda, utilitzant el programa Ripes, aprendrem a veure els diferents tipus d'instruccions presents en el llenguatge assemblador, que es dividiran en grups segons la utilitat que tinguin. Veurem com a partir d'aquestes, les diferents adreces de memòria interactuen entre elles i la variació dels seus valors emmagatzemats.

Es tindrà una part d'exercicis pràctics amb el programa Ripes, i després una sèrie d'exercicis teòric-pràctics, on es treballarà més la part del control d'operacions aritmètiques i lògiques, el control de bits i les diferents representacions d'un nombre o valor, com per exemple la representació de coma flotant, l'ús de màscares i la memòria i el nombre de bits, juntament amb saber que és el que emmagatzema una posició de memòria, i com aquesta pot apuntar a una altra, conegut com a punter.

OBJECTIUS

Trobem que els objectius d'aquesta pràctica són en particular:

- Una introducció amb el simulador Ripes, que inclou aprenentatge del llenguatge ensamblador, funcionament dels diferents processadors de Ripes, com són RSCP i R5SP.
- Aprendre que signifiquen les diferents instruccions i com poden afectar en les adreces de memòria, juntament amb els valor emmagatzemats en aquestes. Observar com un conjunt d'instruccions pot provocar diferents canvis en el tractament de la memòria, i com els valors van variant en la mateixa adreça, es guarden o

carreguen en una altra posició de memòria o com es salta a una altra posició de memòria segons la instrucció feta.

-Comprendre l'analogia entre llenguatge màquina i el llenguatge assemblador. Veure com el llenguatge assemblador

INSTRUCCIONS

Dividim el conjunt d'instruccions en els següents subgrups:

1.Instruccions aritmètic – lògiques

2.Desplaçaments

3.Instruccions d'accés a memòria

4.Instruccions de salt

Instruccions Aritmètic – lògiques :

ADD Rd, Rf1, Rf2 Suma el contingut de Registre font 1 + el contingut de R font 2 i ho guarda en R destí

SUB Rd,Rf1,Rf2 Resta el contingut de R font 1 – el contingut de R font 2 i ho guarda en R destí

ADDI Rd, Rf, Imm Suma el contingut de R font 1 + el nombre Immediat i guarda el resultat en R destí. L'immediat pot ser un valor positiu o negatiu.

AND Rd, Rf1, Rf2 Fa una AND dels continguts de Rf1 i Rf2 i ho guarda en Rd XOR Rd, Rf1, Rf2 Fa una XOR dels continguts de Rf1 i Rf2 i ho guarda en Rd OR Rd, Rf1, Rf2 Fa una OR dels continguts de Rf1 i Rf2 i ho guarda en Rd

ANDI Rd, Rf1, Imm Fa una AND dels continguts de Rf1 i el nombre Immediat Imm i ho guarda en Rd

XORI Rd, Rf1, Imm Fa una XOR dels continguts de Rf1 i el nombre Immediat Imm i ho guarda en Rd

ORI Rd, Rf1, Imm Fa una OR dels continguts de Rf1 i el nombre Immediat Imm i ho guarda en Rd

Desplaçaments:

SLL Rd,Rf1,Rf2 Fa un desplaçament a l'esquerra del contingut que hi ha a Rf1 en nombre de bits que hi ha a Rf2, guardant el resultat en Rd

SRL Rd,Rf1,Rf2 Fa un desplaçament a la dreta del contingut que hi ha a Rf1 en nombre de bits que hi ha a Rf2, guardant el resultat en Rd
SLLI Rd,Rf1,Imm Fa un desplaçament a l'esquerra del contingut que hi ha a Rf1 en nombre de bits que indica el valor immediat Imm, guardant el resultat en Rd
SRLI Rd,Rf1,Imm Fa un desplaçament a la dreta del contingut que hi ha a Rf1 en nombre de bits que indica el valor immediat Imm, guardant el resultat en Rd
SRA Rd,Rf1,Rf2 Fa un desplaçament aritmètic cap a la dreta del contingut que hi ha a Rf1 en nombre de bits que hi ha a Rf2, guardant el resultat en Rd
SRAI Rd,Rf1,Imm Fa un desplaçament aritmètic a la dreta del contingut que hi ha a Rf1 en nombre de bits que indica el valor immediat Imm, guardant el resultat en Rd

Instruccions d'accés a memòria on valor és un nombre o una etiqueta.:

LOAD Rd, Imm(Rf) Desa el contingut que hi ha a la posició de memòria Imm + contingut de Rf en el registre Rd. LA, pseudoinstrucció per assignar una adreça a un registre. Podem parlar de carregar un word LW, carregar 16 bits LH o carregar un byte LB
STORE Rf1, valor(Rf2) Guarda el contingut de Rf1 en la posició de memòria Imm + contingut de Rf2. Igual que en el cas anterior les instruccions poden treballar amb 8 bits SB, 16 bits SH o 32 bits SW
Instruccions de salt:
j posició_Memòria salt incondicional a la posició de memòria posició_Memòria
BEQ Rf1, Rf2, posició_Memòria si el contingut de Rf1 és igual al de Rf2 salta a la posició de memòria posició_Memòria
BEQZ Rf, posició_Memòria salt a la posició de memòria posició_Memòria si Rf és igual a zero
BGT Rf1, Rf2, posició_Memòria si Rf1 > Rf2, salta a la posició de memòria posició_Memòria
BGE Rf1, Rf2, posició_Memòria si Rf1 >= Rf2, salta a la posició de memòria posició_Memòria
BGTZ Rf1, posició_Memòria si Rf1 > 0, salta a la posició de memòria posició_Memòria
BLT Rf1, Rf2, posició_Memòria si Rf1 < Rf2, salta a la posició de memòria posició_Memòria

BLE Rf1, Rf2, posició_Memòria si Rf1 <= Rf2, salta a la posició de memòria posició_Memòria

BLTZ Rf1, posició_Memòria si Rf1 < 0, salta a la posició de memòria posició_Memòria

BLEZ Rf1, posició_Memòria si Rf1 <= 0, salta a la posició de memòria posició_Memòria

BNE Rf1, Rf2, posició_Memòria si Rf1 != Rf2 salta a la posició de memòria posició_Memòria

BNEZ Rf1, posició_Memòria si Rf1!= 0 salta a la posició de memòria posició_Memòria

PART GUIADA

Exercici 1

	nstruccions	Correcte/Incorrecte	Motiu
a	LW a1(R0), a1	incorrecte	lw a1, 0(a1)
b	SW a1,a1(3)	incorrecte	sw a1, 3(a1)
c	BNE 6(t1)	incorrecte	BNE t1, t2, etiqueta
d	ADDI t2, #11, 5(t3)	incorrecte	ADDI t2, t3, 11
e	SUB zero ,a2,a3	correcte	
f	LOAD 3(a0),a1	incorrecte	LOAD a1, 3(a0)

Exercici 2

	Instruccions	x0	a1	a2	a3	M[address1]	M[address2]	PC
a	lw a1, 0(a4)	0	F45A	-----	-----	------	------	+4
b	sw a1,0(a5)	0	-----	-----	-----	------	F45A	+4
c	j etiqueta	0	------	-----	-----	------	-----	et+4
d	addi a3,a2, 11	0	------	A5E3	A5EE	------	-----	+4
e	SUB zero,a2,a3	0	-----	------	------	------	-----	+4
f	srli a1,a1,1		7A2d	------	------	------	------	+4

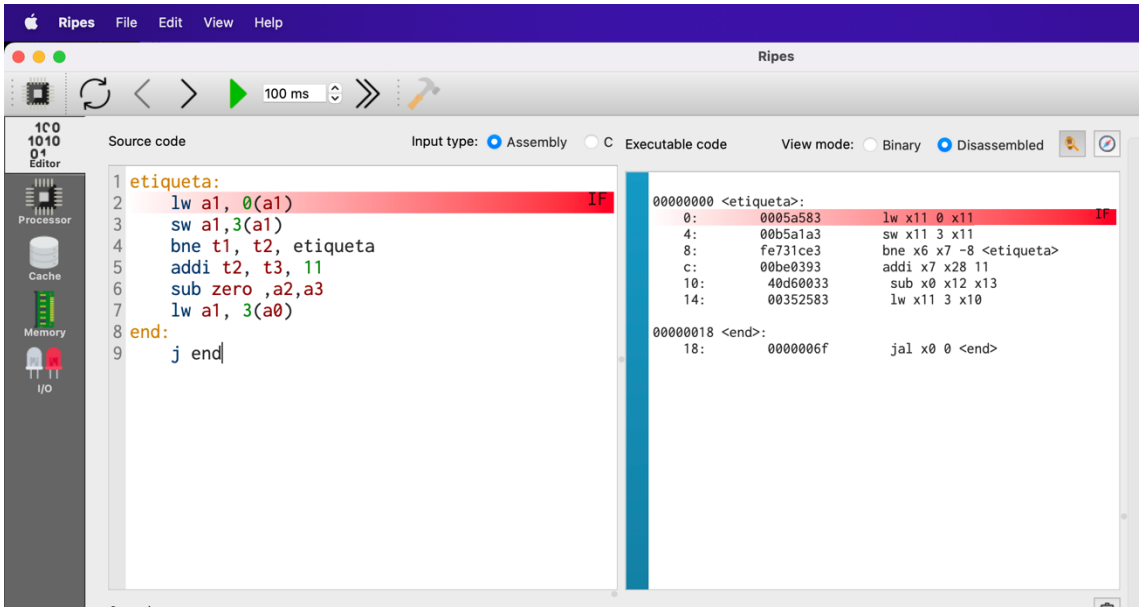
Breu descripció del que fa cada instrucció:
a. guarda en a1 el contingut apuntat per [a4]+0
b. guarda en [a5]+0 el contingut apuntat per a1
c. fa un salt incondicional a la posició de memòria etiqueta
d. fa la suma del contingut de a2 +Immediat=11 i desa el valor en a3
e. fa la resta del contingut de a2- el contingut de a3 i guarda el resultat en zero
f. fa un desplaçament cap a la dreta del contingut de a1 en 1 bit, ja qu el'immediat és 1, i guarda el resultat en a1

Exercici 3

@	M[@]	LLENGUATGE MÀQUINA	LLENG. MAQ. (hex)
.data			
xx:	.word 3		
yy:	.word 5		
oo:	.word 2		
zz:	.word 8		
.text			
00h	la a0 , xx	00010000000000000000010100010111 00000000000001010000010100010011	10000517 00050513
08h	sub a1,a1,a1	01000000101101011000010110110011	40b585b3
0ch	add a2, zero, zero	11111111110001011000100010010011	00000633
10h	loop: addi a7, a1, -4	11111111110001011000100010010011	ffc58893
14h	beqz a7, final	00000000000010001000110001100011	00088c63
18h	lw a3,0(a0)	00000000000001010010011010000011	00052683
1ch	add a2,a2,a3	00000000110101100000011000110011	00d60633
20h	addi a0,a0,4	00000000010001010000010100010011	00450513
24h	addi a1, a1, 1	00000000000101011000010110010011	00158593
28h	j loop	11111110100111111111000001101111	fe9ff06f
2ch	final: sw a2,4(a0)	00000000110001010010001000100011	00c52223

PART PRÀCTICA

Exercici 1



Exercici 2

Pel programa de l'apartat 3 de l'estudi guiat, les posicions de memòria que van des de la xxh a la zzh, ambdues incloses, contenen una seqüència de quatre números 3, 5, 2, 8, emmagatzemats de forma consecutiva. Partint d'aquesta descripció responeu a les següents preguntes:

a) En quina posició de memòria escriu aquest programa el resultat?

Observem que el programa finalitza en l'última línia de codi. Per tant, veiem que donada la instrucció sw a2, 4(a0), això implica que el resultat del programa és el contingut de a2, i escriu aquest en la posició de memòria a0 + 4, és a dir a 4d = 04h, ja que a0 sempre val 0.

b) Què fa el programa?

El programa primer de tot utilitzem el .data per assignar a una determinada posició de memòria un determinat contingut. Seguidament, guarda en 4 adreces de memòria 4 valors diferents, tals que:

xx: .word 3 yy: .word 5 oo: .word 2 zz: .word 8, on la instrucció .word reserva 32 bits de memòria, és a dir 4 bytes.

Seguidament es fa servir la comanda .text per indicar que a partir d'aquella línia de codi comença la memòria del programa. Després s'assigna l'adreça xx al registre a0 amb la instrucció LA i s'inicialitza a1 a 0d, ja que amb la instrucció SUB s'està fent a1-a1, per tant 0d. Després s'utilitza la instrucció ADD per inicialitzar a2, ja que ADD fa zero + zero que òbviament és 0d.

Un cop es tenen inicialitzats a1 i a2, es fa un bucle. En aquest bucle, tenim una sèrie de línies, les quals es repetiran fins que es produeixi una condició. Analitzem l'estructura d'aquest bucle.

Primer de tot, tenim la línia ADDI a7,a1,-4, que el que fa és sumar-li un número immediat (en aquest cas -4) a a1 i guardar aquest resultat en a7. Seguidament tenim la instrucció BEQZ a7, final , la qual serà la línia que ens servirà como a condició i que ens farà sortir del bucle. Aquesta línia fa que si a7 és igual a 0, que salti a la posició de memòria anomenada final. Cal destacar que tant **loop** com **final** són posicions de memòria, les quals el que fan es reservar espai de memòria del programa per aquella posició del programa. La instrucció LW a3 0(a0) fa que el contingut que hi ha a la posició de memòria 0 + a0 sigui guardat a a3. La següent línia, que és ADD a2, a2, a3 fa que la suma a2 + a3 es guardi en a2. La instrucció ADDI a0, a0, 4 fa que es sumi l'immediat 4 amb a0 i que es guardi a a0 el contingut de la posició de memòria a0 + 4. La línia de sota, que és ADDI a1, a1, 1, fa que es sumi l'immediat 1 i a1, i es guardi el resultat a a1. Finalment, tenim un salt incondicional j loop, que fa tornar el programa a la posició de memòria **loop**, sense cap mena de condició. Observem que al cap de 4 iteracions de loop, la instrucció BEQZ confirmarà que a7 és igual a 0 i per tant farà el salt a la posició de memòria final, en la que seguidament hi ha la instrucció SW a2, 4(a0), que el que fa és guardar en la posició 4 + a0 el contingut de a2, finalitzant així el programa.

El que està fent el programa és sumar 3+5+2+8= 18, ja que si ens fixem en LA a0, xx, com que xx té guardat 3, llavors cada vegada que es fa ADD a2, a2 ,a3, el que s'està fent és sumar el contingut de a3, que va canviant per cada iteració, ja que degut a la instrucció

ADDI a0, a0, 4 , el que passa és que es avanci de posició de memòria, que en la primera iteració avançaria a yy, amb contingut 5. Per tant, al cap de 4 iteracions, ja haurem sumat els valors guardats en les adreces de memòria que havíem reservat al principi del codi.

c) Quina sentència de control de flux (en llenguatge d'alt nivell) implementa el programa?

Tal i com s'ha esmentat en l'apartat anterior, veiem que el que estem implementant és equivalent a un **while** o a un **for** en un llenguatge d'alt nivell. Les instruccions que la fan un bucle són BEQZ a7, final i j loop. Veiem que en el cas de la primera instrucció, el que estem fent és una condició amb BEQZ, en la que es comprova si a7 és zero; en cas positiu, es fa un salt a la posició de memòria final, és a dir, es deixa la posició de memòria loop. En cas negatiu, es continuen fent les instruccions. En el nostre cas, cada vegada que la condició era falsa, es sumava el contingut de a3 a a2. Per tant, com en total hem fet 4 iteracions, ja que a1 anava augmentant cada vegada en 1 amb ADDI a1, a1, 1 i després a ADDI a7, a1, -4 cada vegada s'obtenia un resultat més gran, tal que -4, -3, -2, -1, 0. Per tant, aquestes instruccions es repetien perquè si no es complia amb la condició, al final de les instruccions havia un j loop, un salt incondicional que retornava el programa a la posició de memòria loop. Per tant, això en un llenguatge d'alt nivell es podria traduir com: *-while (i<= 4)*

-for(int i = 0; i < 4; i++)

on a7 funcionava com a i en el moment de comparar, i a1 com iterador, ja que a7 cada vegada era més proper a 0 a mesura que a1 augmentava, que ho feia de 1 en 1.

d) Quina és la funció d'a1 al programa? I la d'a0?

La funció de a1 al programa és fer d'iterador, ja que per cada vegada que es fa la instrucció ADDI a1, a1, 1, a1 augmentava en 1. En canvi, la funció de a0 era la de sumar a a2 els valors que havíem guardat en les adreces de memòria xx, yy, oo, zz. Això es feia mitjançant la instrucció ADDI a0,a0,4, que feia guardar a a0 el contingut que havia a la posició de memòria a0 +4.Per tant, la següent iteració a0 tornava a ser 5, però a2 ja era

3, per tant ADD a2, a2, 0(a0) li donaria a a2 el contingut de 8. I així fins acabar el bucle, on sortia amb un valor de 18 en el registre a2.

3. Realitzeu les següents accions sobre el Simulador i expliqueu-ne els resultats:

e) Establiu la relació entre el codi que hi ha a la finestra 'source code' i el que apareix a la finestra 'Executable code'.

Observem que en el programa tenim dues finestres, les quals són la finestra 'source code' i la finestra 'executable code'.
Es pot veure que a la finestra 'source code' es troba el codi en assemblador, és a dir, el codi font original que ha estat escrit i pot ser llegit pel programador.
Mentre que en la finestra de 'executable code' ja es troba el codi compilat a llenguatge màquina que ja pot entendre la màquina.

f) Observeu que inicialment PC=0h (el PC apunta a la següent instrucció executable del programa). Esbrineu com ho fa.

El PC= 0h s'aconsegueix a partir de la instrucció .text, que indica l'inici del programa, i per tant com que encara no s'ha executat cap instrucció en el programa, el PC es situa a 0h.

g) Executeu el programa, instrucció per instrucció, observant el resultat d'executar cada una de les instruccions:

Una vegada hem posat en l'assemblador la instrucció **.text**, comença el programa, després d'haver definit també prèviament les adreces **xx, yy, oo, zz** en el .data. Per tant, tenim el programa següent:

`la a0, xx`

-> Al registre **a0** se li assigna l'adreça de memòria **xx**

`sub a1,a1,a1`

-> Estableix **a1** com a 0, fent **a1-a1** i es guarda a **a1**

```
add a2,zero,zero
```

-> Estableix **a2** com a 0, es fa **zero-zero** i es guarda a **a2**

```
loop:
    addi a7,a1,-4
```

-> Fa la suma **a1 + (-4)** (el valor immediat) i el guarda a **a7**

```
    beqz a7,final
```

-> Comprova si **a7** és igual a 0. En cas positiu, salta a la posició de memòria que hem anomenat final

```
    lw a3,0(a0)
```

-> Guarda el valor de l'adreça **a0 + 0** en **a3**

```
    add a2,a2,a3
```

-> Es fa la suma del valor dels registres **a2** i **a3** i es guarda en **a2**

```
    addi a0,a0,4
```

-> Es fa la suma de **a0+4** i es guarda en en **a0**

```
    addi a1, a1, 1
```

-> Es fa la suma de **a1+1** i es guarda en **a1**

```
    j loop
```

-> Es fa un salt incondicional cap a la posició de memòria loop.

```
final:
    sw a2,4(a0)
```

-> Es guarda el valor de a2 en la posició de memòria a0 + 4

A continuació es mostra una taula amb l'evolució del programa i la modificació que es produeix als registres durant l'execució d'aquest:

Abans de loop				
a0	a1	a2	a3	a7
xx	0	0	No definit	No definit
Iteració 0 de loop				
a0	a1	a2	a3	a7
yy	1	3	3	-4
Iteració 1 de loop				
a0	a1	a2	a3	a7
oo	2	8	5	-3
Iteració 2 de loop				
a0	a1	a2	a3	a7
zz	3	10	2	-2
Iteració 3 de loop				
a0	a1	a2	a3	a7
zz +4	4	18	8	-1
Després de loop				
a0	a1	a2	a3	a7
zz+4	4	18	8	0

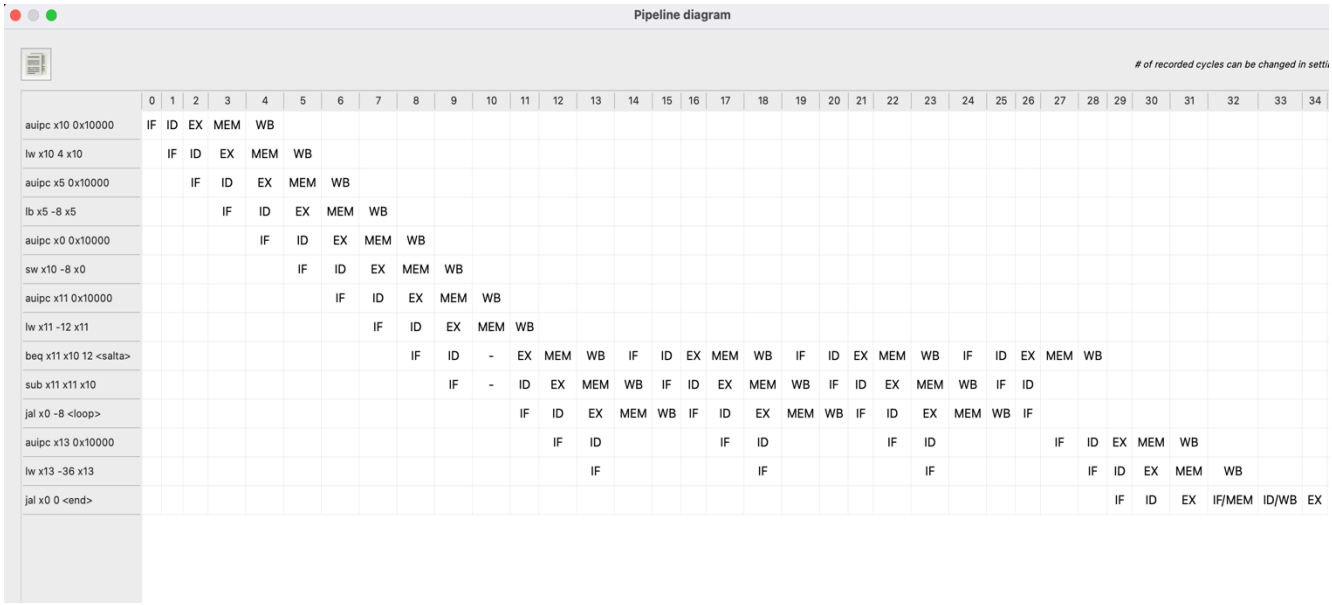
TREBALL A FER A CASA

h) Quan triguen a executar el programa? Per què?

En el primer cas, és a dir amb el processador RSCP, triga 20 cicles. mentre que en el cas del processador R5SP, el programa triga 33 cicles. Això es deu perquè algunes instruccions, tal i com funciona el processador R5SP, mentre que s'estan executant, una altra s'està carregant i una altra es pot descodificar. Per tant, per una mateixa instrucció pot participar en més d'un cicle de rellotge. Per tant, augmenten els cicles de rellotge.

i) Quin és el diagrama d'execució (botó)?

En el cas del processador R5SP, tenim el diagrama:



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34
auipc x10 0x10000	IF	ID	EX	MEM	WB																														
lw x10 4 x10		IF	ID	EX	MEM	WB																													
auipc x5 0x10000			IF	ID	EX	MEM	WB																												
lb x5 -8 x5				IF	ID	EX	MEM	WB																											
auipc x0 0x10000					IF	ID	EX	MEM	WB																										
sw x10 -8 x0						IF	ID	EX	MEM	WB																									
auipc x11 0x10000							IF	ID	EX	MEM	WB																								
lw x11 -12 x11								IF	ID	EX	MEM	WB																							
beq x11 x10 12 <salta>									IF	ID	-	EX	MEM	WB	IF	ID	EX	MEM	WB	IF	ID	EX	MEM	WB	IF	ID	EX	MEM	WB						
sub x11 x11 x10										IF	-	ID	EX	MEM	WB	IF	ID	EX	MEM	WB	IF	ID	EX	MEM	WB	IF	ID								
jal x0 -8 <loop>												IF	ID	EX	MEM	WB	IF	ID	EX	MEM	WB	IF	ID	EX	MEM	WB	IF								
auipc x13 0x10000													IF	ID				IF	ID				IF	ID				IF	ID	EX	MEM	WB			
lw x13 -36 x13														IF					IF					IF					IF	ID	EX	MEM	WB		
jal x0 0 <end>																														IF	ID	EX	IF/MEM	ID/WB	EX

I en el cas del processador RSCP:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
auipc x10 0x10000	•																					
lw x10 4 x10		•																				
auipc x5 0x10000			•																			
lb x5 -8 x5				•																		
auipc x0 0x10000					•																	
sw x10 -8 x0						•																
auipc x11 0x10000							•															
lw x11 -12 x11								•														
beq x11 x10 12 <salta>									•			•			•			•				
sub x11 x11 x10										•			•			•						
jal x0 -8 <loop>											•			•			•					
auipc x13 0x10000																			•			
lw x13 -36 x13																				•		
jal x0 0 <end>																					•	-

j) Quin és el màxim nombre d’instruccions que s’han executat simultàniament?

Veiem que en el cas d'utilitzar el processador RSCP, veiem que, segons observem en el diagrama d'execucions, només s'executa una instrucció cada vegada. En canvi, pel processador R5SP, el seu diagrama indica que s'ha arribat a executar fins 5 instruccions a la vegada.

TEÒRIC-PRÀCTIC

Exercici 1

A·B:

1001 0011

× 0011 1100

1001 0011

0000 0000

1001 0011

0110 1010 1100

A AND B:

1001 0011 AND 0011 1100 = 0001 0000

A + B:

1001 0011

+ 0011 1100

1100 1111

A OR B:

1001 0011 OR 0011 1100 = 1011 1111

Exercici 2

Transforma en representació de coma flotant el següent valor: 3,14 Un cop trobat el valor, feu el procés invers per recuperar el valor

El primer que fem és agafar la part entera del valor, és a dir, el 3 i passar-la a binari, per tant 3= 11 en binari. seguidament, podem aplicar l'algorisme de fer les multiplicacions de la part decimal, tal que 0.14 x 2 = 0.28 (part entera 0), 0.28 x 2 = 0.56 (part entera 0), 0.56 x 2 = 1.12 (part entera 1, agafem part decimal 0.12), 0.12 x 2 = 0.24 (part entera 0), 0.24 x 2 = 0.48 (part entera 0), 0.48 x 2 = 0.96 (part entera 0), 0.96 x 2 = 1.92 (part entera 1, agafem part decimal 0.92), 0.92 x 2 = 1.84 (part entera 1, agafem part decimal 0.84)...

Per tant, la representació en binari del valor 3,14 és 11. 001000111...

Seguidament, el que fem és veure que com és positiu, el primer bit serà un 0 per determinar el signe.

El valor de l'exponent és 127 + 1 = 128, ja que passant a notació científica l'anterior, tenim que 1.1001000111... x 2. Per tant, el valor de l'exponent és només 1 i es fa la suma anterior. Per tant, la mantissa és la part decimal d'aquesta notació científica i per tant, la representació de coma flotant és:

0 10000000 1001000111...

De forma inversa, tenint en compta aquesta representació en coma flotant, el que fem és:

Notem que el signe és positiu, ja que el primer bit és un 0, per tant (-1)^0. Per altra part, tenim que l'exponent és 128, llavors 128-127 = 1. Per altra banda, tenim que el valor de la mantissa és:

1 0 0 1 0 0 0 1 1 1...

2^-1 + 2^-4 + 2^-8 = 0.5 + 1/16 + 1/256 = 0.56640625

Per tant, tenim que:

x = (-1)^0 ·(1.56640625)·2^1 = 3.1328..., que és aproximadament 3.14, degut a la pèrdua de decimals en la representació de coma flotant.

Exercici 3

Feu un programa/pseudocodi que ens indiqui si un determinat valor obtingut per una operació prèvia és positiu o negatiu fent servir màscares i operacions lògiques.

El primer que fem és obtenir el valor obtingut, ja sigui demanat per l'usuari o proporcionat d'alguna forma. Seguidament, el que fem és comparar aquesta valor amb una màscara, per tal de dir si és positiu o negatiu. Utilitzarem l'operació lògica AND (&). Direm que si el valor && MASCARA == 0, llavors el nombre és positiu; en cas contrari, direm que és negatiu. El pseudocodi seria:

```
valor = xxH
MASCARA = 80h ( per exemple, depén del nombre de bits del valor)
if( valor &&. MASCARA == 0):
        'El valor és positiu'
else:
        'El valor és negatiu'
```

Exercici 4

Troba la màscara que has de fer servir per determinar que un número és negatiu. Considerant que el valor està donat en Ca2, passa-ho al valor positiu que correspondria (ex. -7 => 7). Troba vàries solucions al problema.

Tenint en compte que el número/valor que ens donen està en complement a 2 (Ca2), el que busquem per determinar si un número és positiu o no és comparar-lo amb una màscara. En particular, sabent que està en Ca2, només comparant amb el seu signe mòdul ja podem saber si és positiu o no. Per tant, utilitzant el pseudocodi de l'exercici anterior, només cal definir una màscara de la forma MASCARA = 100000...00, és a dir, una màscara de n bits, en funció dels bits del valor que ens passen, amb la condició que el bit de més pes sigui un 1. Per tant, en el moment de comparar, si el AND ens dona diferent de 0, l'única forma per tal d'obtenir aquest resultat és que el primer bit del valor també hagi estat 1, per tant seria un nombre negatiu. En canvi, si ens dóna 0,

sabem que això vol dir que el nombre és positiu perquè l'AND del bit de més pes de valor i MASCARA ha estat 0, per tant diferent valor de bit.
Per passar a valor Ca2, el que hem de fer és negar tots els nombres del valor, i seguidament sumar-li 1. Per tant, agafant l'exemple del -7, sabem que el -7 es representa com 1001 en Ca2 de 4 bits. Per tant el seu negat seria 0110 i posteriorment sumar-li 1, tal que ens quedaria 0111, que és el 7 en decimal.

Exercici 5

La instrucció MOV A, M guarda en el registre A el contingut que hi ha a la posició de memòria M. Indiqueu el contingut del registre A en funció dels possibles valors de M. Quin és el tipus d'adreçament?

Donada la taula que es proporciona l'exercici, observem que en realitat, les dues columnes que es mostren corresponen a l'adreça de M i el valor de M, que a la vegada, per definició de la instrucció MOV A, M, també serà el contingut que tindrà A. Per tant, tenim:

ADREÇA M	CONTINGUT DE A
@6	1234
@5	65434
@4	5435
@3	33223
@2	67
@1	23457
@0	65432

El tipus d'adreçament és indirecte, ja que la mateixa instrucció podrà treballar amb les diferents adreces de memòria de M, per tant, podrà treballar amb els diferents valors que estan a les diferents adreces.

Exercici 6

1- Tenim una memòria de 4kBytes. La unitat de memòria és el word(32 bits). Quants bits necessitem per adreçar-la? Analitzeu i determineu la diferència entre adreça i contingut.

Sabem que 4KBytes = 4 * 1024 Bytes = 4 * 1024 * 8 bits = 32768 bits
Per tant, fent uns càlculs, sabent que 1 word són 32 bits, si fem 32768/32 tenim que necessitem 1024 posicions de memòria. Llavors, veiem que necessitem 10 bits per adreçar una memòria de 4 kBytes, ja que 2^10 = 1024.

La diferència entre adreça i contingut és important per veure com funciona la memòria. En particular, l'adreça és la ubicació física de la posició de memòria, és a dir, el número que identifica la posició. El contingut és el valor emmagatzemat a la posició de memòria.

2-La instrucció LOAD R1, A carrega el contingut que hi ha a la posició de memòria A en el registre R1. Si la memòria és la de l'exercici 1, quants bits ens calen per identificar el valor de A? Si tenim 8 registres de propòsit general R0, R1, ..., R7. Quants bits calen per identificar el registre R1? Si el conjunt d'instruccions total que té aquest processador és de 100 instruccions, quants bits calen per identificar la instrucció LOAD?

Si la memòria és la del exercici 1, hem vist que necessitem 10 bits per adreçar cada posició de memòria. Per identificar el valor de A, necessitem, per tant, 10 bits.

Si tenim 8 registres de propòsit general, cada registre es pot identificar amb 3 bits (ja que 2^3 = 8). Per identificar el registre R1, necessitem el seu número d'índex, que és 1, i per tant, necessitem 3 bits per identificar-lo.

Si el conjunt d'instruccions total que té aquest processador és de 100 instruccions, necessitem 7 bits per identificar cada instrucció, ja que 2^7 = 128, que és suficient per a 100 instruccions. Per identificar la instrucció LOAD, només necessitem tants bits com

siguin necessaris per distingir-la de les altres instruccions. Si suposem que només hi ha una instrucció LOAD, llavors només necessitem 1 bit per identificar-la (per exemple, 0 per a LOAD i 1 per a una altra instrucció).

CONCLUSIONS

Com a conclusions podem extreure que aquesta pràctica ha servit com una introducció al llenguatge ensamblador i com interactua amb la màquina. A més a més, s'ha pogut adquirir coneixement respecte com es comporta un programa de llenguatge de baix nivell segons el processador RSCP i R5SP.

S'ha pogut observar el comportament de les adreces de memòria a partir de les diferents instruccions, i com els valors anaven canviant en les diferents posicions de memòria.

Els exercicis teòric-pràctics han ajudat a adquirir un coneixement extra respecte el tema de la representació dels nombres com complement a 2 o coma flotant, màscares, adreces i valors emmagatzemats, etc.